

Unitree Week 32

Presentation by Mario Ortiz

This Week

- Preparing the Sim2Real Framework based on Sim2Sim implementation

Deployment Software

There are many reasons that though the Sim2Sim task worked, the Sim2Real may not work. Such as:

- Incorrect joint order, motor IDs, signs, IMU frame, quaternion convention, observation order, action scaling...
- Sensor packets
- Threading/race conditions
- Unsafe state transitions
- Poor shutdown behavior
- Unbounded joint commands

Therefore we must look into and account for these problems/find ways to ensure these issues can be dealt with before RL policy deployment.

Our current policy

Trained checkpoint from MJLAB:

“logs/rsl_rl/r1_velocity/2026-08-29_12-50-36/model_2000.pt “

Exported deployment Policy (note ONNX policy uses the same neural network as trained checkpoint):

“deploy/robots/r1/config/policy/velocity/v0/exported/policy.onnx “

The network has

Input: 83 observations

Hidden Layers: $512 \rightarrow 256 \rightarrow 128$

Output: 24 actions

Activation: ELU

Obvservation Normalization **Enabled**

Sim2Sim Deployment Route

MuJoCo R1 Simulation

Simulated LowState

DDS/Unitree SDK2

C++ Deployment FSM

83 Observations

ONNX Policy

24 Actions

Action Scaling/Default Pose

LowCmd

Simulated R1 Motors

Sim2Real Tests

Since at the moment I don't have access to the physical robot, I currently focus on the tasks that do not require it (ie. Unitree R1 files needed for deployment).

Must know:

- Exact R1 model / DoF configuration
- Firmware version(s)
- DDS/network interface
- LowState behavior
- Motor Ids
- IMU convention
- Mode_machine
- Control ownership
- Joint limits
- watchdog/recovery behavior
- Emergency-stop procedure

Automated OffLine Preflight

A file was made to verify as much information as possible **without a robot**. For example, we can verify that specific deployment files exist and software configuration is consistent.

DDS Motor IDs

Verified if the policy's 24 outputs correspond with the correct physical R1 DDS motor IDs.

Deployment map:

[0,1,2,3,4,5,
6,7,8,9,10,11,
12,13,
15,16,17,18,19,
22,23,24,25,26]

Walking Policy controls: 0-24; Head joints (not part of policy): 29,30

Joint-axis and sign conventions

Compared several files such as “r1.xml”, “r1_compiled.xml”, and “scene_r1.xml” of their joints (waist roll, waist yaw, etc.)

- Confirmed that training and deployment simulation use the same axes/signs.

waist roll +X

waist yaw +Z

shoulder pitch +Y

shoulder roll +X

shoulder yaw +Z

elbow +Y

wrist roll +X

83-Element Observation Vector

Compared the training Python Implement with the C++ deployment implementation.

- Base Angular Velocity
- Quaternion
- Projected Gravity
- Commands (Joystick)
- Joint States
- Previous Policy Actions

Observation Normalization

Since the policy was trained using empirical normalization, it was important to verify if the C++ deployment did not normalize the observations too.

Neural-Network Output Path

We compared training (from the file “mjlab/envs/mdp/actions/actions.py”) with the deployment file (“mjlab/envs/mdp/actions/actions.py”) and verified if they both utilized similar training calculations

- Primarily: $q_target = default_q + scale * raw_action$
- The same joint order, action scales, default positions, Kp and Kd

Missing Joint-Target Safety Guard

We found that there is no software guard preventing the robot from exceeding joint limits.

- For example, using the ankle roll
 - Setting the scale = 0.3125 rad/action
 - Model range = ± 0.261 rad
 - If we set action = 1, then $q_target = 0.3125$ rad, which surpasses the model limit.
 - Therefore **additional physical joint-limit protection** must be implemented before hardware deployment.

Read-Only Physical LowState Inspector

Made the file “tools/read_only_lowstate/main.cpp” and the executable “rl_read_only_lowstate” to view interaction with the physical robot.

Right now it only subscribes to “rt/lowstate”, which prints information (message sequence, packet age, mode_machine, motor_state count, IMU quaternion, IMU gyro, q/dq for every motor”

What it will do with the physical robot:

- Publish LowCmd
- Release a motion controller
- Select a mode
- Command a motor

Read-Only LowCmd Range Monitor

“tools/read_only_lowcmd_audit/main.cpp” is a file that subscribes to “rt/lowcmd” and records the following information for each 24 joints:

- Minimum and maximum q target
- Minimum and maximum reconstructed policy action
- Number of target-limit violations

Computes $\text{equivalent_action} = (\text{q_command} - \text{default_q}) / \text{scale}$

Purpose: to ensure the walking policy approaches or exceeds those limits.

Read-Only LowCmd Range Monitor

```
(unitree_rl_mjlab) mario@LAPTOP-9NUH3CE9:~/r1_validation/r1_sim2real_audit_v1$ "$HOME/r1_validation/r1_sim2real_audit_v1/tools/read_only_lowcmd_audit/bin/r1_read_only_lowcmd_audit" lo
1789748986.420344 [0] r1_read_on: selected interface "lo" is not multicast-capable: disabling multicast
READ-ONLY LowCmd audit mode.
DDS interface: lo
Subscribed topic: rt/lowcmd
No publisher created.
No motor commands can be sent by this process.
Press Ctrl+C for the final 24-joint report.

Waiting for rt/lowcmd...
[LowCmd audit] messages=4462 mode_machine=1 joints_ever_outside_model_limit=0 worst_margin=0.106950 rad (right_ankle_roll) nonfinite=0
[LowCmd audit] messages=8908 mode_machine=1 joints_ever_outside_model_limit=0 worst_margin=0.106950 rad (right_ankle_roll) nonfinite=0
[LowCmd audit] messages=13324 mode_machine=1 joints_ever_outside_model_limit=0 worst_margin=0.106950 rad (right_ankle_roll) nonfinite=0
[LowCmd audit] messages=17710 mode_machine=1 joints_ever_outside_model_limit=0 worst_margin=0.106950 rad (right_ankle_roll) nonfinite=0
[LowCmd audit] messages=22071 mode_machine=1 joints_ever_outside_model_limit=0 worst_margin=0.106950 rad (right_ankle_roll) nonfinite=0
[LowCmd audit] messages=26468 mode_machine=1 joints_ever_outside_model_limit=0 worst_margin=0.106950 rad (right_ankle_roll) nonfinite=0
[LowCmd audit] messages=30879 mode_machine=1 joints_ever_outside_model_limit=0 worst_margin=0.106950 rad (right_ankle_roll) nonfinite=0
[LowCmd audit] messages=35156 mode_machine=1 joints_ever_outside_model_limit=0 worst_margin=0.106950 rad (right_ankle_roll) nonfinite=0
[LowCmd audit] messages=39494 mode_machine=1 joints_ever_outside_model_limit=0 worst_margin=0.106950 rad (right_ankle_roll) nonfinite=0
[LowCmd audit] messages=43801 mode_machine=1 joints_ever_outside_model_limit=0 worst_margin=0.106642 rad (right_ankle_roll) nonfinite=0
[LowCmd audit] messages=48107 mode_machine=1 joints_ever_outside_model_limit=0 worst_margin=0.106642 rad (right_ankle_roll) nonfinite=0
[LowCmd audit] messages=52517 mode_machine=1 joints_ever_outside_model_limit=0 worst_margin=0.106642 rad (right_ankle_roll) nonfinite=0
[LowCmd audit] messages=56969 mode_machine=1 joints_ever_outside_model_limit=0 worst_margin=0.106642 rad (right_ankle_roll) nonfinite=0
[LowCmd audit] messages=61410 mode_machine=1 joints_ever_outside_model_limit=0 worst_margin=0.106642 rad (right_ankle_roll) nonfinite=0
[LowCmd audit] messages=65789 mode_machine=1 joints_ever_outside_model_limit=0 worst_margin=0.106642 rad (right_ankle_roll) nonfinite=0
[LowCmd audit] messages=70150 mode_machine=1 joints_ever_outside_model_limit=0 worst_margin=0.106642 rad (right_ankle_roll) nonfinite=0
[LowCmd audit] messages=74499 mode_machine=1 joints_ever_outside_model_limit=0 worst_margin=0.106642 rad (right_ankle_roll) nonfinite=0
[LowCmd audit] messages=78832 mode_machine=1 joints_ever_outside_model_limit=0 worst_margin=0.106642 rad (right_ankle_roll) nonfinite=0
[LowCmd audit] messages=83168 mode_machine=1 joints_ever_outside_model_limit=0 worst_margin=0.106642 rad (right_ankle_roll) nonfinite=0
[LowCmd audit] messages=87514 mode_machine=1 joints_ever_outside_model_limit=0 worst_margin=0.106642 rad (right_ankle_roll) nonfinite=0
[LowCmd audit] messages=91820 mode_machine=1 joints_ever_outside_model_limit=0 worst_margin=0.106642 rad (right_ankle_roll) nonfinite=0
```

Next Steps

More work must be done for Sim2Real, however the physical robot is necessary to proceed. Information (for example, the firmware version, real LowState Data, etc.) will be necessary to do actual deployment to the robot.

Week 31-32 References

https://github.com/unitreerobotics/unitree_rl_mjlab

https://github.com/unitreerobotics/unitree_rl_mjlab/tree/1425b15f73bd4095f0df53709d7c389c3eb9e790

https://github.com/unitreerobotics/unitree_rl_mjlab/blob/main/README.md

https://github.com/unitreerobotics/unitree_mujoco

https://github.com/unitreerobotics/unitree_sdk2_python/blob/master/example/r1/low_level/r1_low_level_example.py

https://github.com/unitreerobotics/unitree_sdk2_python

<https://www.unitree.com/app/R1>

<https://github.com/mujocolab/mjlab/blob/main/docs/source/actions.rst>

https://github.com/mujocolab/mjlab/blob/main/docs/source/entity/entity_data.rst

https://github.com/mujocolab/mjlab/blob/main/src/mjlab/tasks/velocity/velocity_env_cfg.py

https://github.com/leggedrobotics/rsl_rl/blob/main/docs/guide/configuration.rst

https://github.com/leggedrobotics/rsl_rl/blob/main/docs/guide/overview.rst